

CAMPUS COMPLAINT TRACKER

Final Project Blueprint & Technical Specification

Core idea: A campus complaint accountability system where complaints move through defined stages, missed SLAs trigger escalation, and the final resolution can only be closed after student confirmation.

Item	Final decision
Frontend	React.js + Tailwind CSS
Backend	Node.js + Express.js
Storage	JSON files / local server-side data (no database)
Authentication	College-email based prototype authentication
Core engine	Severity + SLA + stage workflow + escalation + student resolution confirmation
Users	Student, Admin/Department, HOD
External reporting	Explicit student choice + structured report / official route

Scope: Functional prototype. The architecture is intentionally database-free now but structured for later migration to persistent storage.

1. Problem Statement

Students may submit campus complaints but still face unclear ownership, delayed action, poor visibility, and complaints being marked resolved even when the issue remains. The system therefore focuses on accountability rather than only complaint submission.

Objective: Give every complaint a clear owner, severity, SLA, progress stages, escalation path, and final student verification.

2. Core Product Principles

- Every complaint gets a unique ID and timestamp.
- Severity and deadlines depend on category/type.
- Admins control internal stages; students do not approve every stage.
- Only the final resolution requires student YES/NO confirmation.
- Admin/HOD can propose a resolution but cannot permanently close it without student confirmation.
- Missed SLAs trigger configured escalation.
- Critical/sensitive complaints use faster or special workflows.
- External reporting is never triggered silently; the student explicitly chooses it.
- Important actions and escalations are recorded in audit history.

3. Users & Roles

Role	Responsibilities
Student	Login, submit, track, receive updates, confirm/reject final resolution, optionally request external reporting.
Admin / Department	Review, assign, process, complete stages, add action/resolution details, monitor SLA.
HOD	Receive escalations, review history, take higher-level action, propose resolution.
System	Generate IDs, deadlines, SLA state, escalation events, notifications and reports.

4. High-Level Architecture

```
STUDENT / ADMIN / HOD BROWSER
|
React.js + Tailwind CSS
|
REST API
|
Node.js + Express.js
|
+-----+-----+-----+
| | |
Auth Complaint Rule/SLA
Roles Service Engine
| | |
+-----+-----+-----+
|
JSON Data
|
users / complaints / rules
notifications / audit logs
```

5. Functional Requirements

ID	Requirement	Priority
FR-01	Approved college-email login.	Must
FR-02	Complaint form: category, description, mobile, relevant location/details.	Must
FR-03	Auto-generate ID, timestamp, severity and SLA deadline.	Must
FR-04	Student complaint timeline and current stage.	Must
FR-05	Admin filters by Normal / High / Critical.	Must
FR-06	Admin marks internal workflow stages complete.	Must
FR-07	Working-day SLA and overdue detection.	Must
FR-08	Automatic escalation based on rules.	Must
FR-09	HOD receives escalated complaints.	Must
FR-10	Admin/HOD submits resolution proposal.	Must
FR-11	Student YES/NO confirmation only at final resolution.	Must
FR-12	NO reopens complaint.	Must
FR-13	Eligible critical/sensitive complaints can expose external reporting choice.	Must
FR-14	Generate structured external report.	Should
FR-15	Audit/history.	Should
FR-16	Responsive UI.	Must

6. Complaint Form

Field	Source	Rule
Complaint ID	System	Auto-generated
Student / college email	Account	Taken from login
Category	Student	Required; drives rules
Description/comment	Student	Required
Mobile number	Student	Required + validation
Location/area	Student	Required where relevant
Severity	System	Derived from category/rules; admin may adjust
Date/time	System	Automatic
SLA deadline	System	Calculated from severity/category and working days
Evidence	Student	Optional prototype feature

7. Suggested Categories & Severity

Category	Examples	Default
Campus maintenance	Fan, light, cleanliness, minor repair	Normal
Hostel	Water, electricity, room, food, security	Normal / High

Category	Examples	Default
Academic	Attendance, marks, result, faculty issue	High
Examination	Exam not allowed, serious access issue	High / Critical
Ragging	Ragging / intimidation	Critical
Harassment	Harassment, stalking, sexual misconduct	Critical
Threat / assault	Threat, physical assault, immediate danger	Critical
Administration	Fees, scholarship, documents	Normal / High
Other	Other grievance	Normal

These are configurable starting defaults, not legal classifications.

8. Rule Engine

The backend Rule Engine decides severity, SLA, escalation, permitted actions and external-report eligibility.

Example configuration:

NORMAL -> department SLA: 7 working days

HIGH -> department SLA: 3 working days

CRITICAL-> immediate/high-priority workflow

Rule sequence:

1. On submission, load category rules.
2. Set severity and calculate deadline.
3. Show On Track / At Risk / Overdue.
4. On SLA breach, create HOD escalation.
5. After HOD period, determine next route from category + severity.
6. External reporting is shown only when configured and only after explicit student choice.
7. Resolution proposal changes status to AWAITING_STUDENT_CONFIRMATION.
8. YES -> CLOSED. NO -> REOPENED.
9. Silence is not automatic confirmation.

9. Status & Stage Model

SUBMITTED -> ASSIGNED -> IN_PROGRESS -> CATEGORY-SPECIFIC STAGES

-> RESOLUTION_PROPOSED -> AWAITING_STUDENT_CONFIRMATION

YES -> CLOSED

NO -> REOPENED -> IN_PROGRESS

Escalation:

IN_PROGRESS -> SLA_BREACHED -> ESCALATED_TO_HOD -> RESOLUTION_PROPOSED

Key rule: Student confirmation is required only for final resolution.

10. Category-Specific Stage Templates

Type	Example workflow
Hostel	Received → Assigned → Inspection → Action/Repair → Resolution Proposed → Student Confirmation
Exam	Received → Examination Department → Documents Verified → Decision/Action → Resolution Proposed → Student Confirmation
Ragging / sensitive	Received → Authorized Authority Notified → Investigation → Action Taken → Resolution Proposed → Student Confirmation
General	Received → Assigned → Investigation/Action → Resolution Proposed → Student Confirmation

11. Escalation Rules

Condition	Action
SLA approaching	Warning to responsible admin/team.
SLA exceeded	Mark overdue and escalate to HOD according to rule.
Critical complaint	Special/fast workflow; do not wait for normal SLA.
HOD period exceeded	Evaluate configured next escalation route.
Eligible critical/sensitive case	Show explicit external reporting option.
Student YES	Generate report/official route after review.
Student NO	Continue escalation workflow.

12. Student Frontend

- Login — approved college email.
- Student Dashboard — counts, severity summary, active/escalated complaints.
- Submit Complaint — structured form.
- My Complaints — list and filters.
- Complaint Details — timeline, SLA, stages and resolution.
- Resolution Confirmation — YES/NO only when awaiting confirmation.
- Notifications — status and escalation updates.
- Profile — student account information.

13. Admin / HOD Frontend

- Admin Dashboard — Normal/High/Critical counts, overdue and escalated cases.
- Complaint Queue — filter by severity, category, status, department and SLA.
- Complaint Details — complete complaint and timeline.
- Stage Action — mark the current internal stage complete with note.
- Resolution Panel — submit resolution proposal; never directly close.
- Escalation Panel — see SLA breach and HOD state.
- HOD Dashboard — escalated cases.
- Reports — complaint counts, SLA and category breakdown.

14. Frontend Structure

```
client/  
  ■■■■ src/  
  ■ ■■■■ components/  
  ■ ■ ■■■■ NavBar.jsx  
  ■ ■ ■■■■ Sidebar.jsx  
  ■ ■ ■■■■ ComplaintCard.jsx  
  ■ ■ ■■■■ StatusBadge.jsx  
  ■ ■ ■■■■ SeverityBadge.jsx  
  ■ ■ ■■■■ ComplaintTimeline.jsx  
  ■ ■ ■■■■ StageProgress.jsx  
  ■ ■ ■■■■ SlaIndicator.jsx  
  ■ ■ ■■■■ ResolutionConfirmation.jsx  
  ■ ■■■■ pages/  
  ■ ■ ■■■■ Login.jsx  
  ■ ■ ■■■■ StudentDashboard.jsx  
  ■ ■ ■■■■ SubmitComplaint.jsx  
  ■ ■ ■■■■ MyComplaints.jsx  
  ■ ■ ■■■■ ComplaintDetails.jsx  
  ■ ■ ■■■■ AdminDashboard.jsx  
  ■ ■ ■■■■ AdminComplaintDetails.jsx  
  ■ ■ ■■■■ HodDashboard.jsx  
  ■ ■■■■ services/api.js  
  ■ ■■■■ context/AuthContext.jsx  
  ■ ■■■■ App.jsx  
  ■■■■ package.json
```

15. Backend Structure

```
server/  
  ■■■■ server.js  
  ■■■■ routes/  
  ■ ■■■■ auth.routes.js
```

```
■ ■■■ complaint.routes.js
■ ■■■ admin.routes.js
■ ■■■ hod.routes.js
■ ■■■ report.routes.js
■■■ controllers/
■■■ services/
■ ■■■ complaint.service.js
■ ■■■ ruleEngine.service.js
■ ■■■ sla.service.js
■ ■■■ escalation.service.js
■ ■■■ notification.service.js
■ ■■■ report.service.js
■■■ middleware/
■■■ data/
■ ■■■ users.json
■ ■■■ complaints.json
■ ■■■ departments.json
■ ■■■ rules.json
■ ■■■ notifications.json
■ ■■■ auditLogs.json
■■■ package.json
```

16. JSON Data Model

```
users.json
{
  "id": "U001",
  "name": "Student Name",
  "collegeEmail": "student@college.edu",
  "mobile": "98XXXXXXX",
  "role": "student"
}

complaints.json
{
  "id": "CMP-2026-00124",
  "studentId": "U001",
  "category": "hostel",
  "description": "No water in Block A",
  "location": "Hostel Block A",
  "severity": "normal",
  "status": "in_progress",
  "createdAt": "2026-10-08T10:32:00",
  "slaDeadline": "2026-10-17T18:00:00",
  "currentStage": "inspection",
  "resolution": null
}
```

17. Important API Endpoints

Method	Endpoint	Purpose
POST	/api/auth/login	Login
GET	/api/complaints	List/filter complaints
POST	/api/complaints	Create complaint
GET	/api/complaints/:id	Complaint details
PATCH	/api/complaints/:id/stage	Complete internal stage
POST	/api/complaints/:id/resolution	Submit resolution proposal
POST	/api/complaints/:id/confirm	Student YES/NO final confirmation
POST	/api/complaints/:id/escalate	Escalation action
GET	/api/notifications	Notifications
POST	/api/reports/external	Generate external report

18. Security & Reliability Rules

- Backend must validate college email and role; do not trust browser-only checks.
- Students cannot modify status or stages.
- Admins cannot directly set CLOSED.
- Only authorized roles can complete a stage.
- Sensitive complaints should have restricted visibility.
- Important changes should be recorded in auditLogs.json.
- Passwords must never be stored as plain text.
- For production, replace JSON storage with persistent database and secure file storage.

19. External Reporting Safety Rule

The prototype must not claim that clicking YES automatically files an FIR/police complaint. Without an official authorized integration, generate a structured report and provide the appropriate official reporting route. The student reviews and explicitly confirms the report before using that route.

20. Dashboard Metrics

Metric	Purpose
Total complaints	Overall workload
Normal / High / Critical	Severity distribution
Open / In Progress	Current workload
Overdue	SLA accountability
Escalated to HOD	Higher-level intervention
Awaiting student confirmation	Resolution verification workload
Reopened	Issues not actually resolved
Closed	Student-confirmed resolutions
Category breakdown	Recurring campus problems

21. Complete End-to-End Example

1. Student logs in with approved college email.
2. Student submits "No water in Hostel Block A".
3. Backend creates CMP-2026-00124.
4. Category = Hostel; severity = Normal.
5. System calculates the working-day SLA.
6. Admin sees the complaint under Normal.
7. Admin completes stages: Assigned → Inspection → Action/Repair.
8. If SLA is exceeded, system marks overdue and escalates to HOD.
9. HOD/authorized admin records action taken.
10. Admin/HOD submits "Resolution Proposed".
11. Student sees the proposed resolution.
12. YES → CLOSED.
13. NO → REOPENED and returned to the workflow.
14. For an eligible critical/sensitive case, external reporting can be explicitly selected.
15. The system generates a structured report containing complaint details, timeline, actions and escalation history.

22. Project File Blueprint

```
Campus-Complaint-Tracker/  
■■■■ client/ # React + Tailwind  
■■■■ server/ # Node + Express  
■ ■■■■ routes/  
■ ■■■■ controllers/  
■ ■■■■ services/  
■ ■■■■ middleware/  
■ ■■■■ data/  
■■■■ docs/  
■ ■■■■ requirements.md  
■ ■■■■ architecture.md  
■ ■■■■ rules.md  
■ ■■■■ api.md  
■■■■ .env.example  
■■■■ .gitignore  
■■■■ README.md
```

23. Development Phases

Phase	Build
1	React/Tailwind + Express setup and folders
2	Prototype college-email authentication and roles
3	Complaint form + JSON storage + ID generation
4	Student dashboard + details + timeline
5	Admin dashboard + severity + stage progression
6	SLA/working-day Rule Engine
7	Automatic HOD escalation + HOD dashboard
8	Resolution proposal + student YES/NO + reopen
9	External eligibility + report generation
10	Notifications, audit history, responsive polish and testing

24. Final Product Definition

Campus Complaint Tracker is a responsive React/Tailwind + Node/Express prototype focused on complaint accountability. Its differentiator is category-based severity and SLA rules, stage-by-stage admin progress, automatic escalation, HOD intervention, and a mandatory student confirmation before final closure.

Production note: The no-database version is for demonstration/prototyping. A real college deployment would require persistent storage, secure authentication, file storage, notification infrastructure, institutional policy configuration and authorized external reporting integrations.